

Detailed Use Case Specifications

This document lists the nineteen formalized use cases for the data curation platform. The corresponding user story is reproduced under each specification heading for reference.

Authentication and Access

UC-01: Register and Log In

US-01: As a curator, I want to register and log in to my account so that I can keep my workspaces private and access them across sessions.

Title: Register and Log In **Actor:** Curator

Preconditions:

- The user has reached the login screen of the deployed instance.

Basic Flow:

1. The user selects the registration tab and enters a username, optional email, and password.
2. The system validates the input format (username 2-50 characters, password 4-128 characters, email well-formed when provided).
3. The system creates the user account and signs an access and refresh token pair.
4. The system stores the tokens in the auth store and redirects to the workspace list.

Alternative Flows:

- **1a. Existing account:** The user selects the login tab, enters credentials, and the system verifies them against the stored bcrypt hash before issuing tokens.
- **2a. Validation failure:** The system displays an inline error and remains on the login screen; no account is created.
- **2b. Username taken:** The system rejects registration with an HTTP 409 message; the user may choose another username.
- **3a. Standalone mode:** No backend call is made; authentication is local and tokens are not used.

Postconditions:

- The user is authenticated and has an active session.
- The auth store holds the user's identifier, username, and role.

UC-02: Access Administrator-Only Screens

US-02: As an administrator, I want to access administrator-only configuration screens so that I can manage system settings without exposing them to curators.

Title: Access Administrator-Only Screens **Actor:** Administrator

Preconditions:

- The administrator is authenticated and the auth store reports the role as `admin`.

Basic Flow:

1. The administrator opens the Services entry from the sidebar.
2. The system loads the current endpoint configuration and the most recent health probe results from the backend.
3. The system renders the endpoint table with the refresh and edit controls available.
4. The administrator can edit endpoint URLs and adapters and persist the changes.

Alternative Flows:

- **1a. Non-admin user:** The Services panel is still reachable, but the system renders the endpoint table in read-only mode; the edit and save controls are inaccessible, so configuration changes are restricted to administrators.
- **2a. Backend unreachable:** The system displays a notification and falls back to the bundled default endpoint configuration.

Postconditions:

- The administrative configuration panel is visible to the user.
- Endpoint metadata and health status are displayed.
- Edit and save operations are gated to users carrying the `admin` role.

Workspace Management

UC-03: Create New Workspace

US-03: As a curator, I want to create a new workspace so that I can isolate and organize my active document annotations.

Title: Create New Workspace **Actor:** Curator

Preconditions:

- The curator is authenticated and is on the workspace list.

Basic Flow:

1. The curator selects the new-workspace control.

2. The system prompts for a workspace name.
3. The curator enters a name and confirms.
4. The system validates the name (non-empty after trim, unique among the curator's workspaces) and creates an empty workspace through the workspace application service.
5. The system persists the workspace through the storage gateway and opens the editor on the new workspace.

Alternative Flows:

- **4a. Empty or whitespace-only name:** The system rejects the input with an inline validation message.
- **4b. Duplicate name:** The system flags the duplicate and the curator must choose another name before confirming.
- **5a. Persistence failure:** The system surfaces an error notice with a retry action; the workspace is not added to the list.

Postconditions:

- A new workspace exists in the curator's workspace list.
- The new workspace is persisted to the configured storage backend.

UC-04: Retrieve Existing Workspace

US-04: As a curator, I want to retrieve an existing workspace so that I can resume my previous post-editing sessions.

Title: Retrieve Existing Workspace **Actor:** Curator

Preconditions:

- The curator is authenticated and at least one workspace exists.

Basic Flow:

1. The curator selects a workspace from the list.
2. The system loads the serialized workspace document through the storage gateway.
3. The system hydrates the session store with the document and resets the undo history.
4. The system opens the editor on the loaded workspace.

Alternative Flows:

- **2a. Document not found:** The system displays an error notice and returns to the workspace list.
- **2b. Document fails to load:** The system displays an error notice with a retry action and does not partially hydrate the session.

Postconditions:

- The selected workspace is the active session.

- The undo history is cleared.

UC-05: Delete Workspace

US-05: As a curator, I want to delete a workspace so that I can keep my workspace list manageable.

Title: Delete Workspace **Actor:** Curator

Preconditions:

- The curator is authenticated and at least one workspace exists.

Basic Flow:

1. The curator selects the delete action on a workspace row.
2. The system displays a confirmation dialog naming the workspace.
3. The curator confirms the deletion.
4. The system deletes the workspace through the storage gateway and removes it from the workspace list.

Alternative Flows:

- **3a. Cancel:** The curator dismisses the dialog and the workspace remains.
- **4a. Deletion failure:** The system surfaces an error notice with a retry action; the workspace remains in the list.

Postconditions:

- The selected workspace is removed from the list and from the storage backend.

UC-06: Import Raw Text

US-06: As a curator, I want to import raw text into a workspace so that the document becomes available for subsequent NLP pipeline processing.

Title: Import Raw Text **Actor:** Curator

Preconditions:

- The curator is creating a new workspace, or has just created one whose editor is still empty.

Basic Flow:

1. The curator initiates workspace creation. The new-workspace dialog offers an optional file-import field that accepts a plain-text file (size limit one megabyte).
2. The curator either attaches a `.txt` file before confirming, or confirms the dialog without a file.

3. If a file was attached, the system reads its contents and populates the new workspace's source text. Otherwise, the workspace opens with an empty editor placeholder reading "Insert your document text here..."
4. If the workspace is empty, the curator pastes the source text directly into the placeholder.
5. The system stores the source text in the session and marks it dirty; auto-save persists it at the next debounce window (UC-07).

Alternative Flows:

- **2a. File exceeds the configured size limit:** The system rejects the file with a size-limit message; the curator may paste the contents instead.
- **4a. No paste performed:** The placeholder remains visible and no source text is registered until the curator pastes or types.

Postconditions:

- The workspace contains the supplied source text.
- The session is dirty and an auto-save is scheduled.

UC-07: Automatic State Save

US-07: As a curator, I want the system to automatically save the complete application state so that I do not lose complex annotations in the event of a browser crash.

Title: Automatic State Save **Actor:** System (initiated by any curator mutation)

Preconditions:

- The curator has a workspace open and the session has transitioned from clean to dirty.

Basic Flow:

1. The session store emits a state transition that the auto-save subscriber receives.
2. The subscriber classifies the change as either a typing burst or a discrete action.
3. For a typing burst, the subscriber arms a three-second idle timer; for a discrete action, a one-second timer.
4. On timer expiry, the system serializes the workspace and persists it through the storage gateway.
5. The system marks the session clean.

Alternative Flows:

- **3a. Maximum-delay cap reached:** A thirty-second hard-cap timer fires and triggers a save regardless of ongoing activity.

- **4a. Save fails:** The system displays an error notice with a retry action and re-arms the debounce timer.
- **4b. Snapshot drift:** The session changed while the save was in flight; the synchronizer re-arms the timer to capture the most recent state.
- **1a. Forced save (navigation, logout, workspace switch):** The unsaved-changes guard bypasses the debounce and persists the session immediately.

Postconditions:

- The persisted workspace document reflects the most recent committed session state.

Text Segmentation

UC-08: Automatic Text Segmentation

US-08: As a curator, I want to request automatic text segmentation so that I can obtain an initial, machine-generated structural division of the document.

Title: Automatic Text Segmentation **Actor:** Curator

Preconditions:

- The curator has a workspace open with non-empty source text.
- The segmentation NLP endpoint is configured.

Basic Flow:

1. The curator triggers the segmentation action.
2. The system dispatches the source text to the segmentation endpoint through the API service.
3. The endpoint returns segment boundaries as character offsets.
4. The system replaces the segment list in the session and renders boundary markers in the editor.

Alternative Flows:

- **1a. Segmentation already present:** The system blocks the action and surfaces a warning notice (“Document has already been segmented”); the curator must clear or manually adjust the existing segments before requesting automatic segmentation again.
- **3a. Endpoint failure:** The system displays an error notice with a retry action; segments remain unchanged.
- **3b. Endpoint returns empty:** The system displays a notice; the curator may segment the document manually using the merge and boundary-drag controls described in UC-09.

Postconditions:

- The workspace contains an ordered segment list aligned with the source text.

UC-09: Adjust Segment Boundaries

US-09: As a curator, I want to manually adjust segment boundaries so that I can correct illogical sentence divisions without breaking the underlying text offsets.

Title: Adjust Segment Boundaries **Actor:** Curator

Preconditions:

- The curator is logged in and has an active workspace open.
- The NLP API segmentation has been requested, and the results are rendered in the editor.

Basic Flow:

1. The system displays the segmented text with visible boundary markers between segments.
2. The curator identifies a boundary that requires correction.
3. The curator selects the boundary marker and drags it to the correct character position within the text.
4. The system recalculates segment lengths and updates the rendered text in real time.
5. The system persists the updated segmentation topology to the workspace state.

Alternative Flows:

- **3a. Merge segments:** The curator selects the boundary between two adjacent segments and removes it entirely.
- **3b. Split segment:** The curator selects a delimiter character inside an existing segment and confirms the *Split segment here* action; the system inserts a boundary at that position.
- **5a. Revert:** The curator discards the change, and the boundary reverts to its original position.

Postconditions:

- The workspace state reflects the updated segmentation boundaries.
- All downstream annotations (NER tags, translation mappings) have their character offsets recalculated and remain correctly aligned with the corrected segment boundaries.

UC-10: Split and Merge Segments

US-10: As a curator, I want to split and merge segments at any point during the curation workflow so that I can iteratively refine the document structure as I discover boundary issues during annotation.

Title: Split and Merge Segments **Actor:** Curator

Preconditions:

- The curator has a workspace open with at least one segment.

Basic Flow:

1. The curator selects a delimiter character inside a segment (split) or selects the join control on a segment boundary (merge).
2. The system invokes the action guard to detect cross-language translation gaps in the affected segments.
3. The curator confirms the action when prompted.
4. The system updates the segment list, recalculates downstream span offsets, and adjusts translation entries as required.
5. The auto-save mechanism persists the change.

Alternative Flows:

- **2a. Translation gap detected:** The action guard presents a resolution dialog enumerating the affected languages and segments; the curator resolves each gap (by translating the missing segments or deleting the affected translations) or cancels the action.
- **3a. Cancel:** The action is aborted and the segment topology is unchanged.

Postconditions:

- The segment list reflects the requested split or merge.
- All downstream annotations and translation entries are consistent with the new topology.

Named Entity Recognition

UC-11: Request NER Analysis

US-11: As a curator, I want to request NER analysis so that I can minimize the manual effort of manually identifying and categorizing entities.

Title: Request NER Analysis **Actor:** Curator

Preconditions:

- The curator has a workspace open with at least one segment containing text.
- The NER NLP endpoint is configured.

Basic Flow:

1. The curator triggers NER, either on the active segment or globally over the workspace.

2. The system dispatches the segment text (or each segment in turn for global NER) to the NER endpoint.
3. For each returned span, the system applies the conflict resolution algorithm against existing user and API spans (UC-13).
4. The system stores the surviving spans on the active layer and renders them as inline highlights.

Alternative Flows:

- **2a. Endpoint failure on a single iteration (global NER):** The system records the failure, continues with remaining segments, and presents a partial-success notice with the count of succeeded and failed segments.
- **2b. Total failure:** The system displays an error notice with a retry action; existing spans are unchanged.
- **3a. No conflicts:** All returned spans are accepted silently without prompting the curator.

Postconditions:

- The active layer's API span list reflects the new NER output, with conflicts resolved per the curator's choices.

UC-12: Manually Edit Named Entity Annotations

US-12: As a curator, I want to manually edit, delete, or add named entity annotations at any point so that I can correct or augment the automatic NER results based on my domain expertise.

Title: Manually Edit Named Entity Annotations **Actor:** Curator

Preconditions:

- The curator has a workspace open and the editor is rendered.

Basic Flow:

1. The curator selects a text range in the editor.
2. The system displays an entity type picker.
3. The curator selects an entity type.
4. The system creates a user span at the selected coordinates and renders it as an inline highlight.

Alternative Flows:

- **1a. Click on an existing span:** The system opens the edit menu allowing the curator to change the entity type or delete the span.
- **3a. Selection overlaps an existing API span:** The system attaches the user span. The next NER run resolves the overlap through the conflict resolution algorithm (UC-13), at which point the curator decides whether to retain the user span or accept the API span.

- **4a. Delete action:** The system removes the span from the active layer and adds its `start:end:entity` key to the deleted-API-key set, so that re-running NER does not resurrect the same span at the same coordinates and entity type.

Postconditions:

- The active layer’s user span list and deleted-API-key set reflect the curator’s edits.

UC-13: Resolve NER Annotation Conflict

US-13: As a curator, I want a dedicated interface to resolve annotation conflicts so that I can maintain consistency between my manual corrections and newly fetched API suggestions.

Title: Resolve NER Annotation Conflict **Actor:** Curator

Preconditions:

- The curator has an active workspace with at least one segment processed by the NER API.
- The state management system has detected a positional or categorical conflict between the curator’s local annotations and the incoming API response.

Basic Flow:

1. The system presents a view, showing the curator’s local annotation alongside the API’s suggested entity.
2. For each conflict, the curator selects one of two resolution strategies: keep the local annotation or accept the API suggestion.
3. The curator confirms the resolution for the current conflict.
4. The system applies the selected state transition and advances to the next conflict.
5. The curator repeats steps 1-4 for any remaining conflicts in the document.
6. The system persists the reconciled annotation state to the workspace.

Alternative Flows:

- **2a. No conflicts:** No conflicts exist in the workspace, the system notifies the curator, and the resolution dialog remains inactive.

Postconditions:

- All annotation state conflicts within the processed segments are resolved.
- The workspace annotation state is internally consistent and contains no remaining conflicts.

Translation Post-Editing

UC-14: View and Edit Machine Translations

US-14: As a curator, I want to view and edit machine-generated translations so that I can correct contextual inaccuracies and preserve historical authenticity.

Title: View and Edit Machine Translations **Actor:** Curator

Preconditions:

- The curator has a workspace open.
- At least one translation exists for a target language, either previously generated or imported.

Basic Flow:

1. The curator selects the target language tab on a segment.
2. The system displays the translated text in an editable region.
3. The curator edits the translation.
4. The system updates the segment-translation entry and sets the per-segment edit flag.

Alternative Flows:

- **2a. Segment not yet translated:** The system displays an empty region and offers a per-segment translate action.
- **4a. Cancel:** The curator reverts the change before the auto-save fires.

Postconditions:

- The translation entry for the target language and segment reflects the curator's edit.
- The per-segment edit flag is set, marking the segment as manually edited for downstream consumers (export annotations, re-translation skip logic in UC-15).

UC-15: Re-translation Edit Preservation

US-15: As a curator, I want to be warned before re-translation overwrites my manually edited translations so that I can decide whether to discard my edits or cancel the re-translation request.

Title: Re-translation Edit Preservation **Actor:** Curator

Preconditions:

- The curator has a workspace with a translation in at least one target language.
- At least one segment in that language already carries a translation, manually edited or otherwise.

Basic Flow:

1. The curator triggers per-segment re-translation (the Sync action in the segment header) on a segment that carries the *Edited* badge.
2. The system presents a confirmation dialog naming the target language and warning that the manual edit will be overwritten.
3. The curator confirms the dialog.
4. The system fetches a fresh translation from the API, overwrites the existing segment translation entry, and clears the per-segment edit flag.

Alternative Flows:

- **2a. Cancel:** The curator dismisses the dialog. The existing translation and its edit flag remain unchanged; no API call is issued.
- **1a. Per-segment re-translation of an unedited segment:** When the targeted segment does not carry the *Edited* badge, the system skips the confirmation step, fetches a fresh translation, and overwrites the existing entry. No prior manual edits are at risk at this granularity.
- **1b. Workspace-level re-translation over partial coverage:** When the curator triggers translation of the entire workspace into a target language that already has partial coverage, the system iterates segments and skips any that already carry a translation in that language, regardless of whether the per-segment edit flag is set; only untranslated segments are dispatched to the API. Existing entries (including edited ones) and their edit flags remain untouched. No confirmation dialog is presented at this granularity since no edits are at risk of being overwritten.
- **4a. API failure on a single iteration (workspace-level):** The system records the failure, continues with remaining segments, and presents a partial-success notice with the count of succeeded and failed segments.
- **4b. Total API failure:** The system displays an error notice with a retry action; the existing translation state is unchanged.

Postconditions:

- Per-segment re-translation (confirmed): the targeted entry reflects the fresh API response and its edit flag is cleared.
- Per-segment re-translation (cancelled): the targeted entry and its edit flag remain unchanged.
- Workspace-level re-translation: existing segment translations and their edit flags are preserved; only previously untranslated segments receive new content.

Thesaurus Tagging

UC-16: Automatic Thesaurus Tagging

US-16: As a curator, I want to request automatic thesaurus tagging for segments so that I obtain initial topic classifications without manual effort.

Title: Automatic Thesaurus Tagging **Actor:** Curator

Preconditions:

- The curator has a workspace open with at least one segment.
- The classification NLP endpoint is configured.

Basic Flow:

1. The curator triggers semantic tagging on the active segment or globally.
2. The system dispatches the segment text to the classification endpoint.
3. The endpoint returns suggested thesaurus tags.
4. The system attaches the suggested tags to the segment or workspace as appropriate and updates the tag panel.

Alternative Flows:

- **2a. Segment text empty:** The action is skipped for that segment and reported in the partial-success notice.
- **3a. Endpoint failure:** The system displays an error notice with a retry action; existing tags are unchanged.
- **4a. Duplicate tag suggestion:** The system silently skips tags already attached at the same scope.

Postconditions:

- The tag list of the affected segment or workspace reflects the new suggestions.

UC-17: Manual Tag Assignment

US-17: As a curator, I want to assign tags to segments either from the historical thesaurus or as custom free-text tags so that I can capture both standardized topics and ad-hoc annotations relevant to the document.

Title: Manual Tag Assignment **Actor:** Curator

Preconditions:

- The curator has a workspace open with at least one segment.

Basic Flow:

1. The curator opens the tag panel and selects a target segment or the workspace as the tag scope.
2. The curator types a query into the tag input.
3. The system queries the thesaurus search worker and renders matched entries.
4. The curator selects an entry, or chooses the custom-tag affordance to attach a free-text label.
5. The system attaches the tag at the chosen scope.

Alternative Flows:

- **3a. No thesaurus matches:** The system offers the custom-tag affordance directly.
- **4a. Tag already attached at this scope:** The system blocks the attachment and surfaces a notice.
- **5a. Persistence failure:** The system surfaces an error notice; the tag is not attached.

Postconditions:

- The selected tag is attached to the chosen scope.

Export**UC-18: Export Workspace**

US-18: As a curator, I want to export my curated workspace as JSON or PDF so that I have a portable record of the annotation work for archival, sharing, or external processing.

Title: Export Workspace **Actor:** Curator

Preconditions:

- The curator has access to at least one workspace, either active or in the workspace list.

Basic Flow:

1. The curator selects the export action on a workspace row or from the active editor.
2. The system displays a format chooser (JSON or PDF).
3. The curator selects a format.
4. The system loads the workspace document through the storage gateway, transforms it through the export contract, generates the artifact in-browser, and triggers a download.

Alternative Flows:

- **3a. Cancel:** The format chooser is dismissed and no artifact is generated.
- **4a. Workspace cannot be loaded:** The system displays an error notice; no download is triggered.
- **4b. PDF rendering failure:** The system surfaces an error notice with a retry action; the curator may retry, cancel, or choose the JSON format manually.

Postconditions:

- The exported artifact is available to the curator as a local download.
- The internal workspace state is unchanged.

Administration

UC-19: Configure NLP API Endpoints

US-19: As an administrator, I want to configure NLP API endpoint parameters so that I can adapt the platform to backend infrastructure changes without modifying the source code.

Title: Configure NLP API Endpoints **Actor:** Administrator

Preconditions:

- The administrator is authenticated and the auth store reports the role as **admin**.

Basic Flow:

1. The administrator opens the configuration panel.
2. The system loads the current endpoint configuration and the latest health probe results.
3. The administrator edits the URL or adapter selection of an endpoint.
4. The administrator saves the change.
5. The system writes the configuration through the configuration service.
6. The administrator triggers the refresh control to re-run the health probe and verify that the new endpoint is reachable; the table updates with the latest status.

Alternative Flows:

- **3a. Refresh:** The administrator triggers an immediate re-probe without modifying the configuration; the system displays the updated status table.
- **4a. Validation failure:** The system rejects the save with an inline message; the configuration is unchanged.
- **5a. Persistence failure (platform mode):** The system surfaces an error notice with a retry action; the configuration is unchanged.

Postconditions:

- The endpoint configuration is updated and persisted.
- Subsequent NLP requests for the affected service type use the new URL or adapter selection.